Cambridge (CIE) A Level Computer Science

Your notes

Structured Programming

Contents

- Procedures & Functions

Your notes

What are functions and procedures?

- Functions and procedures are a type of **sub program**, a **sequence of instructions** that **perform a specific task** or **set of tasks**
- Procedures and functions are **defined at the start of the code**
- Sub programs are often used to **simplify** a program by **breaking it into smaller, more manageable parts**
- Sub programs can be used to:
  - **Avoid duplicating code** and can be reused throughout a program
  - **Improve** the **readability** and **maintainability** of code
  - Perform **calculations**, to **retrieve data**, or to **make decisions** based on input
- **Parameters** are **values** that are **passed into a sub program**
  - Parameters can be variables or values and they are located **in brackets** after the name of the sub program
  - Example: FUNCTION TaxCalculator(pay,taxcode) **OR** PROCEDURE TaxCalculator(pay,taxcode)
- Sub programs **can have multiple parameters**
- To use a sub program you '**call**' it from the main program

What's the difference between a function and procedure?

- A **Function returns a value** whereas a procedure does not

Procedures

- Procedures are defined using the **PROCEDURE** keyword in pseudocode
- A procedure can be written as:

Pseudocode	
PROCEDURE <identifier>() <statement(s)> ENDPROCEDURE	PROCEDURE <identifier>(<param1>:<data type>, <param2>:<data type>...) <statement(s)> ENDPROCEDURE

- The identifier is used to **call the procedure**
- To call a procedure, it can be written as:

```
CALL <identifier>()
CALL <identifier>(Value1,Value2,...)
```


© 2025 Save My Exams, Ltd.

- When parameters are used, value1, value2... must be of the correct data type and in the same sequence

Your notes

Example code

```
PROCEDURE GreetUser(Name : STRING)
   OUTPUT "Hello, ", Name, "!"
ENDPROCEDURE

DECLARE UserName : STRING

OUTPUT "Enter your name: "
INPUT UserName

CALL GreetUser(UserName)
```

- PROCEDURE GreetUser defines the procedure, which takes one parameter: Name
- The procedure prints a personalised greeting
- The main program asks for the user's name and then **calls** the procedure using CALL

Functions

- Functions are defined using the **FUNCTION** keyword in pseudocode or **def** keyword in Python
- Functions **always return a value** so they **do not** use the CALL function, instead they are called **within an expression**
- A function can be written as:

Pseudocode	
FUNCTION <identifier> RETURNS <data type> <statement(s)> ENDFUNCTION	FUNCTION <identifier>(<param1>:<data type>, <param2> <statement(s)> ENDFUNCTION

Example code

```
FUNCTION calculate_area(length: INTEGER, width: INTEGER)
  area ← length * width
  RETURN area
ENDFUNCTION

# Output the value returned from the function
OUTPUT(calculate_area(5,3))
```

- Defines a **function** called calculate_area
- It takes **two parameters**: length and width, both of type INTEGER
- A local variable area is declared and assigned the result of length × width
- The function **returns** the result to wherever it was called from

Parameter passing

Your notes

What is parameter passing?

- Parameter passing is the method by which **values or references** are given to procedures or functions so they can **use or modify data during execution**
- When writing **procedures**, you can control how parameters are passed using:
  - BYVAL – pass **a copy** of the value (default behaviour)
  - BYREF – pass **the actual variable**, so changes inside the procedure affect the original

Keyword	What it does
BYVAL	Passes a **copy** of the value (no change to original)
BYREF	Passes a **reference** to the variable (can be modified)

- If **no keyword** is used, it is assumed to be BYVAL
- **Functions** should **not use** BYREF **parameters** – only **procedures** should

Example – Swapping two values (Procedure with BYREF)

```
PROCEDURE Swap(BYREF X : INTEGER, Y : INTEGER)
  DECLARE Temp : INTEGER
  Temp ← X
  X ← Y
  Y ← Temp
ENDPROCEDURE
```

- X is passed **by reference**, so it **can be changed**
- Y is passed **by value**, so any change to Y inside the procedure **does not affect** the original value

Worked Example

A video-conferencing program supports up to six users. Speech from each user is sampled and digitised (converted from analogue to digital). Digitised values are stored in array Sample.

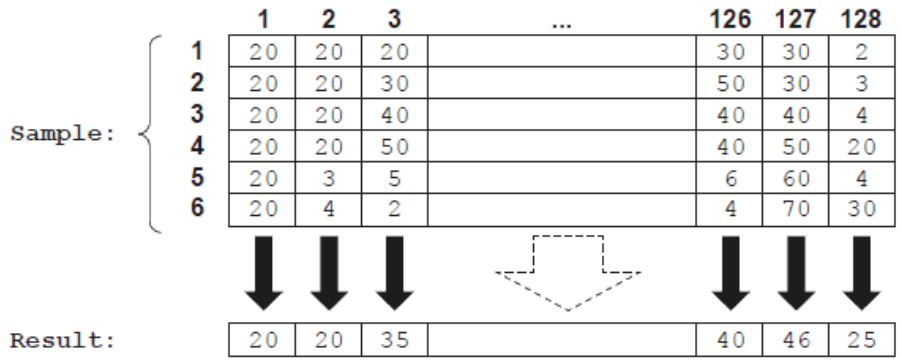
The array Sample consists of 6 rows by 128 columns and is of type integer. Each row contains 128 digitised sound samples from one user.

The digitised sound samples from each user are to be processed to produce a single value which will be stored in a 1D array Result of type integer. This process will be implemented by procedure Mix().

A procedure Mix() will:

- calculate the average of each of the 6 sound samples in a column
- ignore sound sample values of 10 or less
- store the average value in the corresponding position in Result
- repeat for each column in array Sample

The diagram uses example values to illustrate the process:



Write pseudocode for procedure Mix().

Assume Sample and Result are global. **[6]**

Answer

```
PROCEDURE Mix()
  DECLARE Count, Total ThisNum : INTEGER
  DECLARE ThisUser, ThisSample : INTEGER

  FOR ThisSample ← 1 TO 128
    Count ← 0
    Total ← 0
    FOR ThisUser ← 1 TO 6
      IF Sample[ThisUser, ThisSample] > 10 THEN
        Count ← Count + 1
        Total ← Total + Sample[ThisUser, ThisSample]
      ENDIF
    NEXT ThisUser
    Result[ThisSample] ← INT(Total / Count)
  NEXT ThisSample

ENDPROCEDURE
```

- Declaration **and** initialisation before inner loop of Count **and** Total [1 mark]
- Outer Loop for 128 iterations [1 mark]
- Inner loop for six iterations [1 mark]
- Test for sample > 10 **in a loop** [1 mark]
- and if true sum Total and increment Count [1 mark]
- Calculate average value and assign to Result array **after inner loop and within outer loop** [1 mark]
- Use of INT()/ DIV to convert average to integer [1 mark]

Your notes